

Suspicious Activity Detection System

Security Log Analyzer for Failed-Login Detection

Design & Documentation

Item	Detail
Implementation language	Python 3 (standard library only: os, sys)
Input	Plain-text security log file (one event per line)
Processing	Count 401 / 403 failures per IP; flag IPs with more than 5 failures
Output	Console report: failure counts, flagged IPs, first and last failure time
Main components	get_filename, readlogfile, parse_line, trackfailures, detect_suspicious_ips, print_report, main

Contents

- 1. Problem Statement
- 2. Objectives
- 3. Functional Requirements
- 4. Non-functional Requirements
- 5. System Architecture Diagram
- 6. Process Flow / Workflow Diagram
- 7. UML Diagrams (Use Case, Class / Component, Sequence)
- 8. Database / Storage Design (ER Diagram, Schema Design)
- 9. Dataset Description, Model Selection Rationale and Evaluation Methodology
- 10. Limitations and Future Enhancements

1. Problem Statement

Servers, web applications and network services record every authentication attempt in log files. Many failed logins (status 401 Unauthorized or 403 Forbidden) coming from the same IP address are a classic sign of a brute-force or credential-stuffing attack. Reading large log files by hand is slow, error-prone and does not scale, so suspicious sources are easily missed.

This project provides a lightweight command-line tool that reads a security log, counts failed login attempts for each IP address, flags the addresses whose failures exceed a configurable threshold, and prints a clear report including when the first and last failure occurred.

2. Objectives

- Automate the detection of repeated failed logins from a log file.
- Count failed attempts (status codes 401 and 403) for every unique IP address.
- Flag any IP address whose failures exceed the threshold (THRESHOLD = 5, i.e. 6 or more failures).
- Give investigators context by reporting the number of failures and the first and last failure timestamps for each flagged IP.
- Handle missing files, empty files, blank lines and malformed lines without crashing.
- Offer flexible input: command-line argument, interactive prompt, or a default sample file.
- Keep the design modular, readable and dependency-free so it is easy to maintain and extend.

3. Functional Requirements

ID	Requirement	Implemented by
FR1	Accept the log file name from a command-line argument or an interactive prompt; use the default file sample_security_log.txt when nothing valid is supplied.	get_filename()
FR2	Resolve the entered path as given or relative to the script folder, and strip surrounding quotes and spaces.	get_filename()
FR3	Read the file line by line, ignore blank lines, and print an error message if the file does not exist.	readlogfile()
FR4	Split each line into date, time, IP address and status code; skip lines with fewer than four fields.	parse_line()
FR5	Count failed attempts per IP address when the status code is in FAILED_CODES (401, 403).	trackfailures()
FR6	Store the full timestamp (date + time) of every failure for each IP address.	trackfailures()
FR7	Flag every IP address whose failure count is greater than THRESHOLD.	detect_suspicious_ips()
FR8	Print a report with the number of unique IPs with failures, the failure count per IP, and for each flagged IP the count, first failure and last failure.	print_report()
FR9	Print 'No suspicious activity detected.' when no IP is flagged, and print a message and stop when the log has no usable data.	print_report(), main()

4. Non-functional Requirements

Category	Requirement	How it is met
Performance	Process large logs quickly.	Single pass over the data; dictionary lookups are $O(1)$, so total time is $O(n)$.
Reliability	Never crash on bad input.	<code>FileNotFoundError</code> is caught; malformed lines return <code>None</code> and are skipped; blank lines are removed.
Usability	Easy to run for non-experts.	Works with no arguments (default file), a CLI argument, or a prompt; readable formatted report.
Maintainability	Easy to read and modify.	Small single-purpose functions; tunable values held in constants (<code>FAILED_CODES</code> , <code>THRESHOLD</code>).
Portability	Run on any OS with Python 3.	Standard library only; <code>os.path</code> used for paths.
Security	Do not alter or expose log data.	Log file opened read-only; no network access; output limited to the console.
Extensibility	Allow new rules or outputs.	Detection, parsing and reporting are separated, so each can be replaced independently.

5. System Architecture Diagram

The tool is a small layered pipeline. The control layer obtains the file name, the data access layer reads the log, the processing layer parses lines, counts failures and flags suspicious IPs, and the presentation layer prints the report. Intermediate results are kept in memory only.

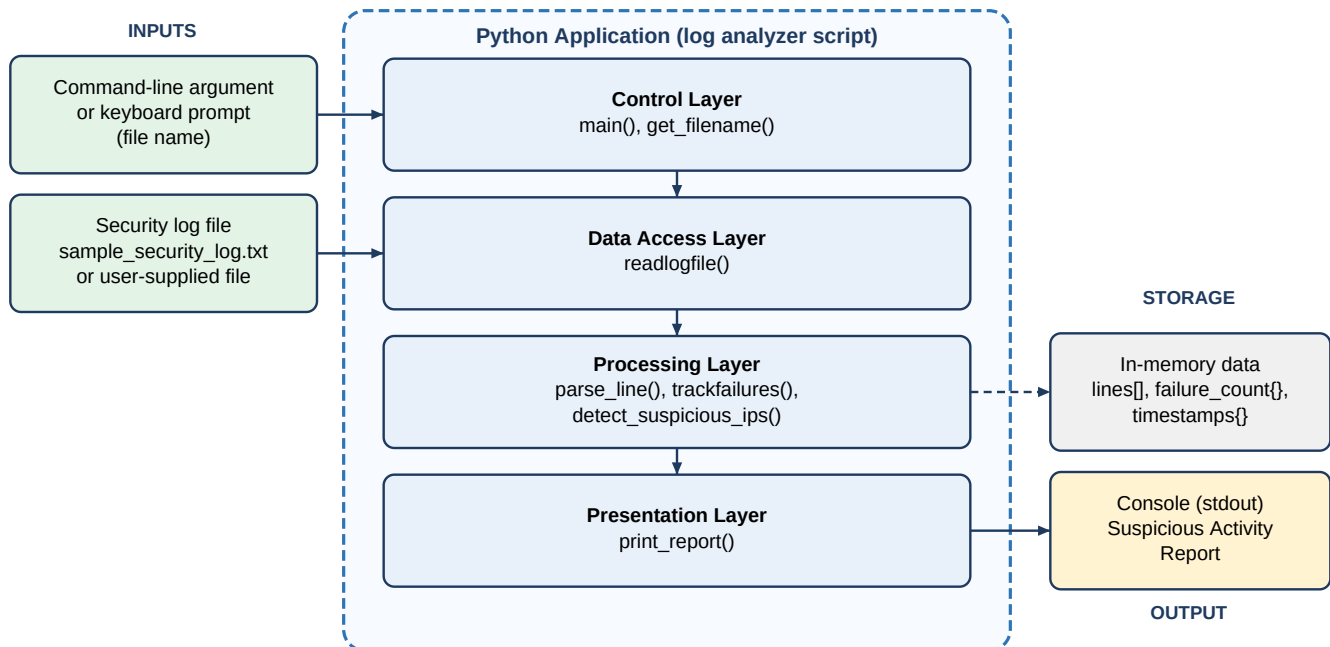


Figure 1: Layered system architecture

6. Process Flow / Workflow Diagram

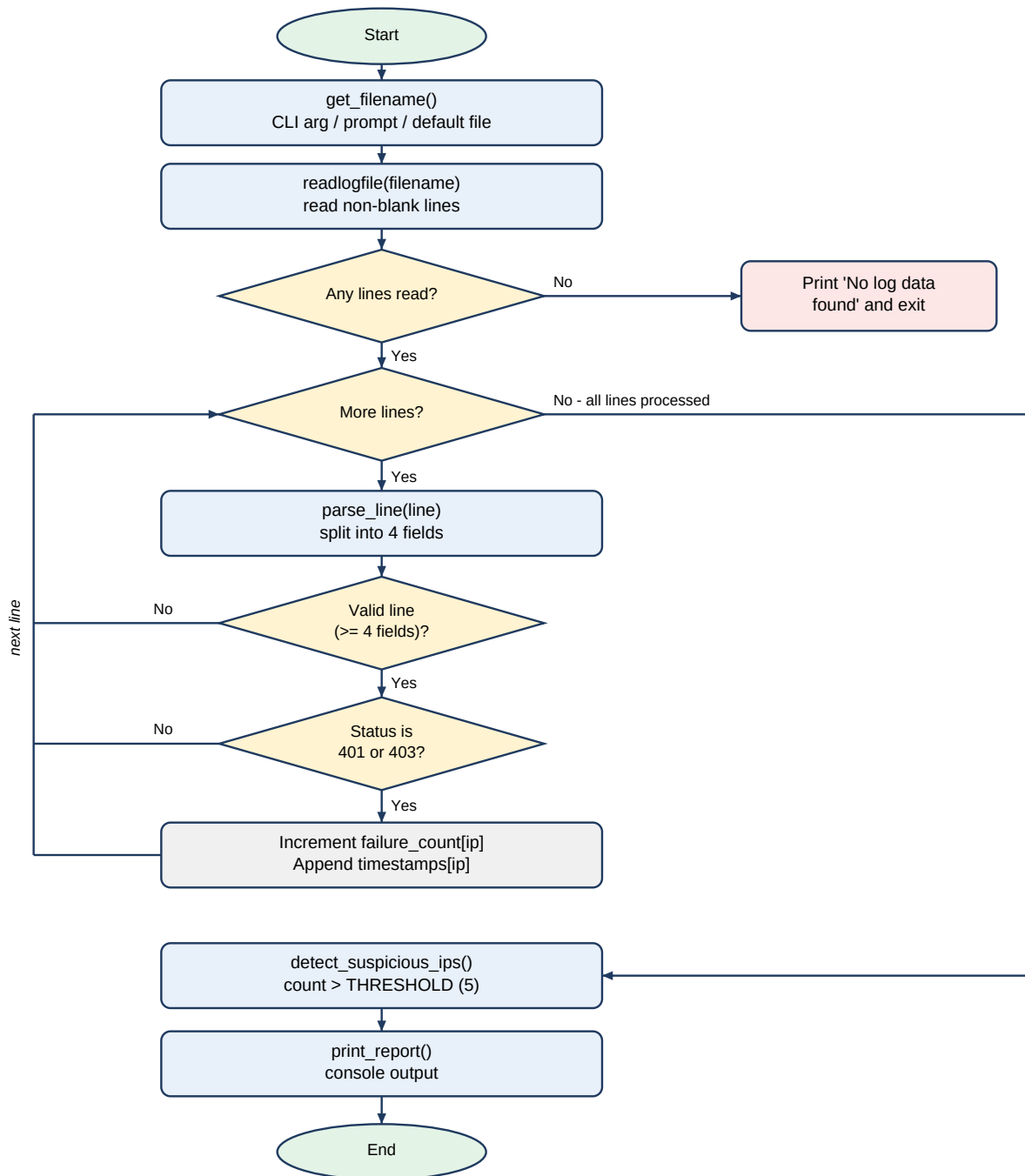


Figure 2: Workflow of the log analysis process

7. UML Diagrams

7.1 Use Case Diagram

The Security Analyst is the only actor. Selecting the file can fall back to the default sample file (extend), and running the analysis includes reading and validating lines, counting failures and flagging suspicious IPs.

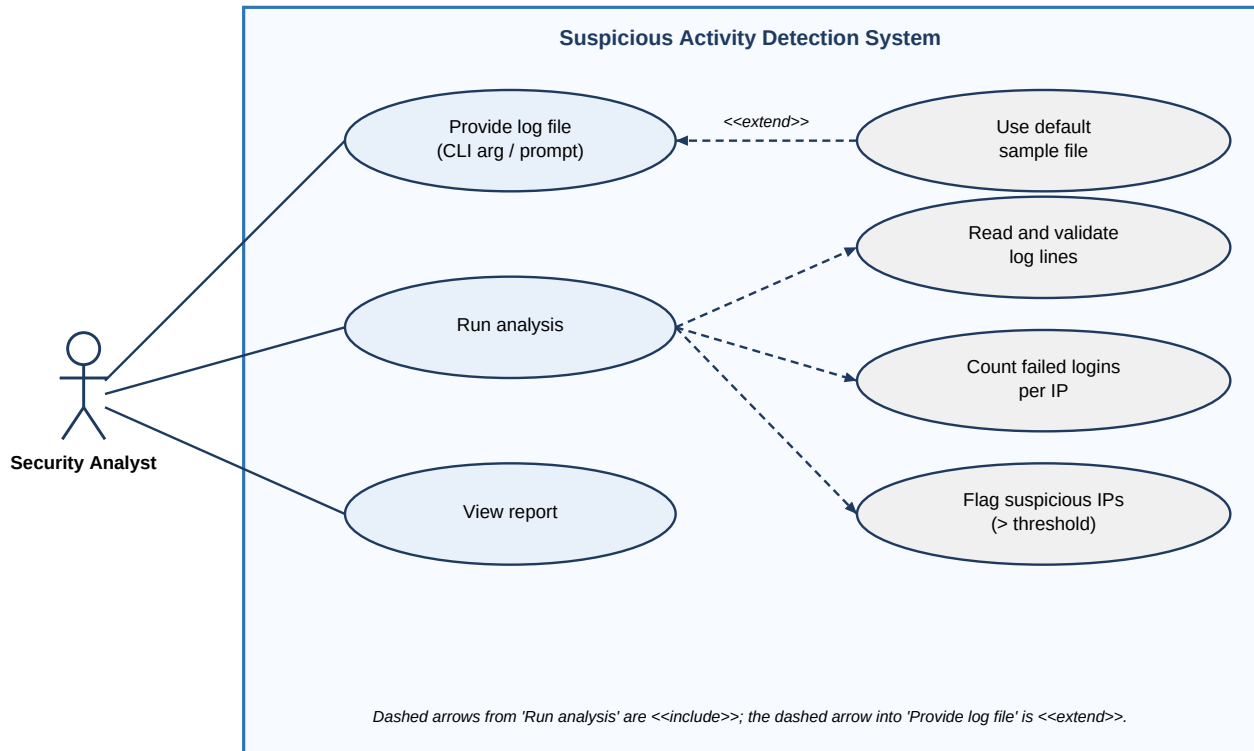


Figure 3: Use case diagram

7.2 Class Diagram and Component Diagram

The implementation is procedural, so the class diagram shows the script as a module with its constants and operations, plus the two dictionaries that carry the results. The component diagram shows how the functions depend on each other and on external resources.

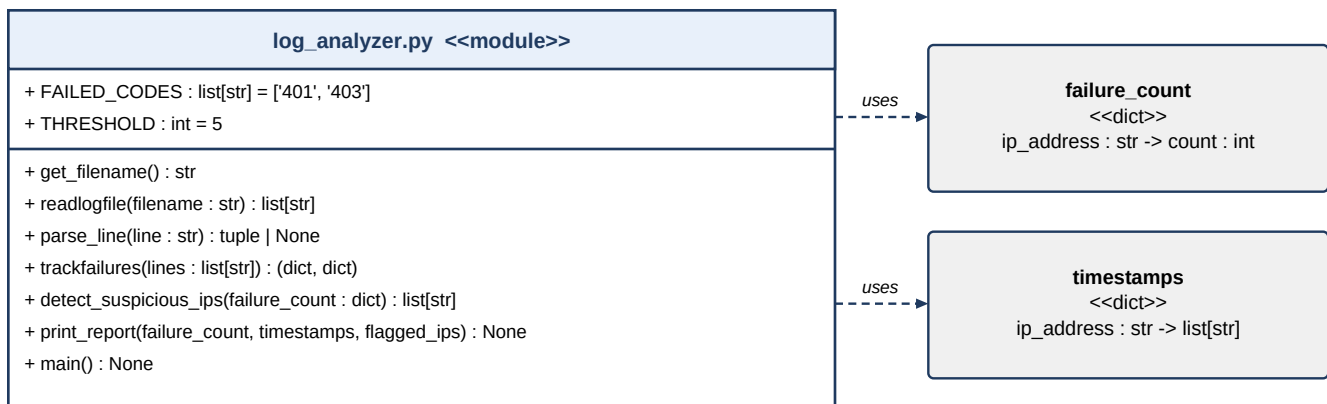


Figure 4: Class diagram (module view)

Solid arrows: calls / data flow. Dashed arrows: reads configuration.

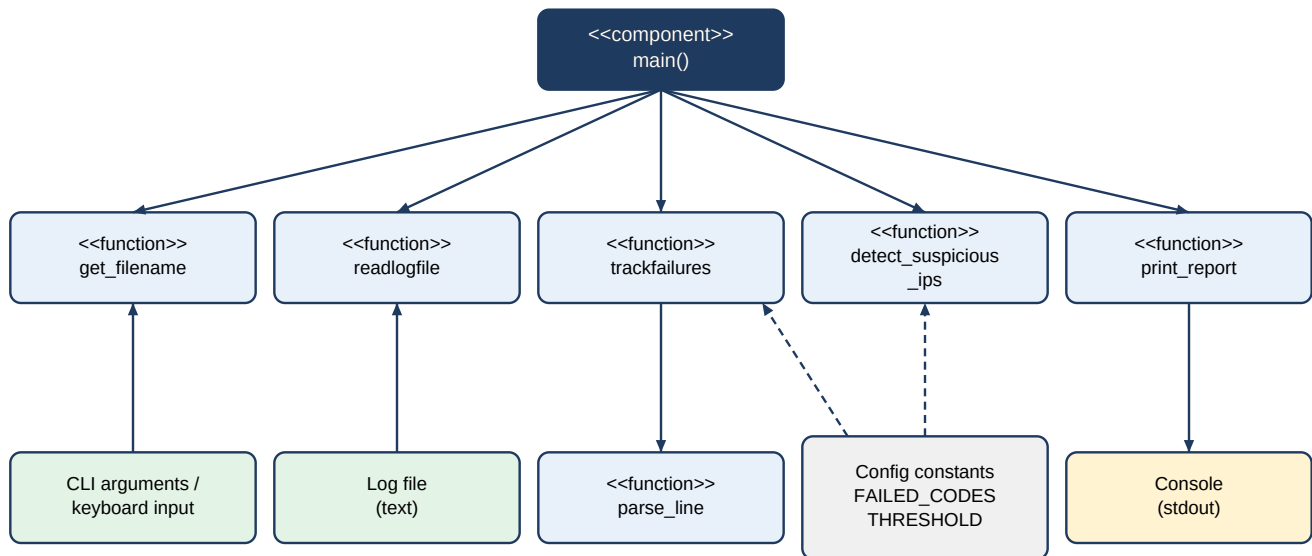


Figure 5: Component diagram

7.3 Sequence Diagram

The sequence below shows a typical run in which the user is prompted for a file name.

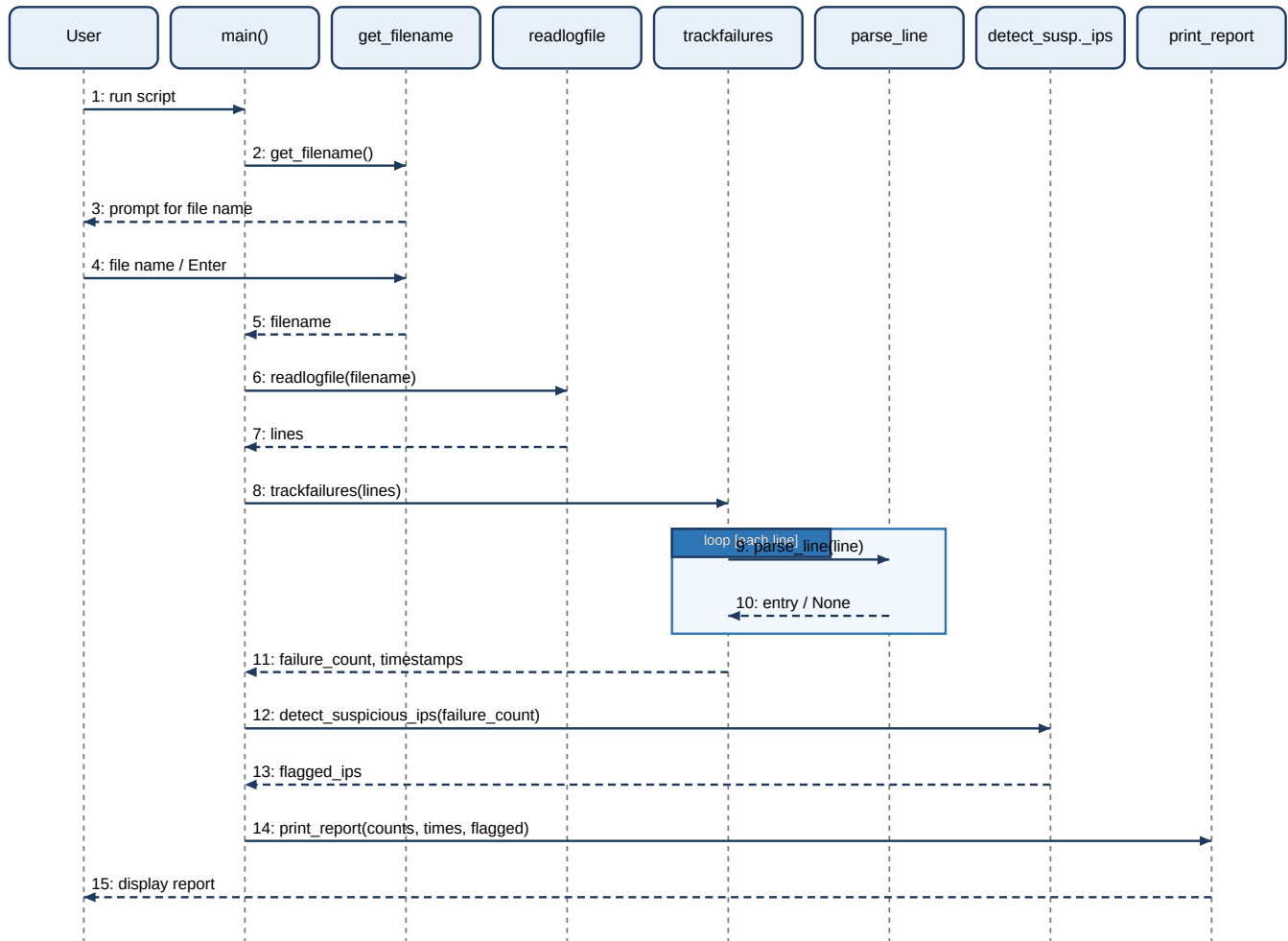
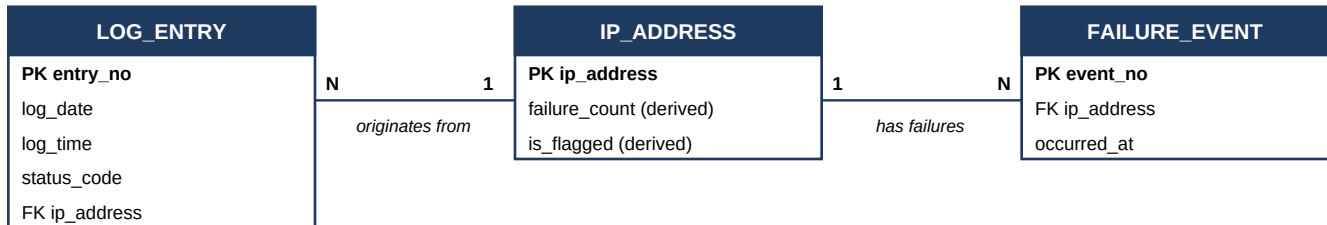


Figure 6: Sequence diagram of a complete analysis run

8. Database / Storage Design

The current program does not use a database. Input comes from a text file and all intermediate data is held in Python lists and dictionaries for the duration of the run. The ER diagram and schema below describe the logical data model and how it could be persisted in a relational database if the tool is extended.

8.1 ER Diagram



Notation: 1 = one, N = many. PK = primary key, FK = foreign key.

Derived attributes are computed at run time (count of FAILURE_EVENT rows; count > THRESHOLD).

A FAILURE_EVENT exists only for log lines whose status code is 401 or 403.

Figure 7: Entity-relationship diagram (logical model)

8.2 Schema Design

(a) In-memory structures used by the program

Structure	Type	Key	Value	Created by
lines	list[str]	index	One non-blank log line	readlogfile()
failure_count	dict[str, int]	IP address	Number of failed attempts	trackfailures()
timestamps	dict[str, list[str]]	IP address	List of 'date time' strings, in file order	trackfailures()
flagged_ips	list[str]	index	IP addresses with count greater than THRESHOLD	detect_suspicious_ips()

(b) Logical relational schema (optional persistence, not used by the current script)

```
CREATE TABLE ip_address (  
    ip_address TEXT PRIMARY KEY  
);  
CREATE TABLE log_entry (  
    entry_no INTEGER PRIMARY KEY,  
    log_date TEXT NOT NULL,  
    log_time TEXT NOT NULL,  
    status_code TEXT NOT NULL,  
    ip_address TEXT NOT NULL REFERENCES ip_address(ip_address)  
);  
CREATE TABLE failure_event (  
    event_no INTEGER PRIMARY KEY,  
    ip_address TEXT NOT NULL REFERENCES ip_address(ip_address),  
    occurred_at TEXT NOT NULL  
);  
CREATE VIEW ip_failure_summary AS  
SELECT ip_address,  
    COUNT(*) AS failure_count,  
    COUNT(*) > 5 AS is_flagged,  
    MIN(occurred_at) AS first_failure, -- assumes ISO-style timestamps  
    MAX(occurred_at) AS last_failure  
FROM failure_event  
GROUP BY ip_address;
```

9. Dataset Description, Model Selection Rationale and Evaluation Methodology

This project is rule-based rather than machine-learning based, so no model is trained. The sections below document the data the tool works on, why a simple rule was chosen, and how correctness is evaluated.

9.1 Dataset Description

Aspect	Description
Source	Text log file, by default sample_security_log.txt in the script folder, or any file supplied by the user.
Record format	One event per line, whitespace-separated: date time ip_address status_code . Extra fields after the fourth are ignored.
Example (illustrative)	2025-01-10 08:15:23 192.168.1.10 401
Relevant values	Status codes 401 and 403 count as failed logins; all other codes are ignored.
Preprocessing	Leading/trailing spaces removed, blank lines dropped, lines with fewer than four fields skipped.
Labels / ground truth	None in the file; expected results for evaluation are computed by hand for test logs.

9.2 Model Selection Rationale

- **Chosen approach: fixed-threshold rule** (failures > 5 per IP). It is deterministic, easy to explain to auditors, needs no training data and runs in linear time.
- **Alternatives considered:** machine-learning anomaly detection (needs labelled or large historical data and is harder to explain) and sliding-window rate limiting (more precise, but needs reliable timestamp parsing; listed as a future enhancement).
- The threshold and the list of failure codes are constants, so the rule can be tuned without changing the logic.

9.3 Evaluation Methodology

Correctness is evaluated with black-box test logs whose expected output is calculated manually, plus boundary and error cases:

#	Test case	Expected result
T1	One IP with 7 failures (401/403)	IP flagged; count 7; first and last timestamps shown
T2	One IP with exactly 5 failures (boundary)	Counted but not flagged (rule is strictly greater than 5)
T3	One IP with exactly 6 failures	IP flagged
T4	Lines with success or other codes (e.g. 200, 404)	Ignored; not counted
T5	Line with fewer than four fields	Skipped without error
T6	Blank lines mixed into the log	Removed by readlogfile()
T7	Empty file	'No log data found' message and the program stops
T8	Non-existent file given as argument	'File not found' error, then 'No log data found'
T9	Non-existent name typed at the prompt	Message printed and the default sample file is used
T10	Several IPs, none above threshold	'No suspicious activity detected.'

Additional measures: compare the reported counts with an independent count (for example a spreadsheet or grep), and, if labelled attack data becomes available, compute precision and recall of the flagged IPs. Runtime can be measured on progressively larger logs to confirm linear scaling.

10. Limitations and Future Enhancements

Observation	Suggested improvement
Failures are counted over the whole file, not within a time window.	Use a sliding window (for example more than 5 failures in 10 minutes).
First and last failure are taken from file order, which assumes the log is chronological.	Parse timestamps with datetime and take the true minimum and maximum.
Status codes are compared as strings and IP addresses are not validated.	Validate the IP format (ipaddress module) and normalise status codes.
Only FileNotFoundError is handled when reading.	Also handle PermissionError and decoding errors.
Results appear only on the console.	Add export to CSV/JSON and optional alerts (e-mail, firewall block list).
Threshold and status codes are hard-coded.	Read them from command-line options or a configuration file.
The input prompt text contains a typo ('prass').	Change the wording to 'press enter'.